

Module 9

Topics: Using datasets with multiple observations per subject

Files to download: *patients.sas7bdat, mod9.sas7bdat, flbygroup.sas7bdat, patients_part2.sas7bdat, screener_data.sas7bdat*
 SAS Library files: *SASHELP.PRDSALE*

9.1 - Using datasets with multiple observations for the same person

Until now, we have been using datasets that contained only one observation for each unique individual. Datasets that contain multiple observations for the same person can provide a lot more information but they may require a bit more data manipulation for proper analysis.

This section will introduce some of the basic techniques that you may find useful in working with these kinds of datasets. First, we will be working with a dataset called *patients.sas7bdat* that stores data on patients collected during clinic visits held over the past several years. Let's take a look at the dataset using the viewtable.

```
LIBNAME NICOLE 'H:\AEPI537D';
```

The first variable in the dataset is ID and it is unique for each patient. In addition, the first 3 digits code for the clinic that the patient visits. The next field is VISDATE and this stores the date of the clinic visit. The database is currently sorted on RES_ZIP which is the first 3 digits of the zip code in which the patient lives. You can see that there is more than one record for several of the patients living in each zip code area. Let's create a temporary dataset, sort it by id and then look at this new dataset. Remember to close the viewtable before trying to run the program.

```
DATA TEMP;
    SET NICOLE.PATIENTS;
RUN;

PROC SORT DATA=TEMP;
    BY ID;
RUN;
```

Now you can see that there are several visits for each patient. The first things that you would probably want to know are the number of unique subjects in your dataset and the distribution of number of visits for these patients. One way to get this information would be to first create a visit variable and give it the value 1 for every observation. Then you could use PROC SUMMARY to get the sum of this visit variable for each patient in the following way.

```
DATA TEMP2;
    SET TEMP;
    VISIT=1;
RUN;
```

AEPI537D

```
PROC SUMMARY DATA=TEMP2;
  CLASS ID;
  VAR VISIT;
  OUTPUT OUT=TEMP3 SUM=TOTVISIT;
RUN;
```

Use the viewtable to take a look at TEMP3. The output dataset that you created has three variables: ID with each unique ID listed only once; _TYPE_ which is a variable that keeps track of the combinations of the class variables; _FREQ_ which is the count of the occurrences of the class variable; and TOTVISIT which is the variable name that we assigned to store the sum. The value stored for the _TYPE_=0 level of the class is always the grand total, in this case 2,148 which is the total number of visits (and the total number of observations in the dataset).

If you scroll to the bottom of the dataset you will find that there are 471 observations. Since the first observation does not have an ID value (because it is the grand total) this means that the dataset contains data on 470 unique ID values (study subjects). We can look at the distribution of the number of visits among the study subjects with a SAS frequency procedure limited to values of TOTVISIT for which _TYPE_=1.

```
PROC FREQ DATA=TEMP3;
  TABLES TOTVISIT;
  WHERE _TYPE_=1;
RUN;
```

Here is the output of the run:

The FREQ Procedure				
	Frequency	Percent	Cumulative Frequency	Cumulative Percent
1	4	0.85	4	0.85
4	214	45.53	218	46.38
5	224	47.66	442	94.04
6	28	5.96	470	100.00

The frequency ranges from 1 to 6 visits with most study subjects having either 4 or 5 visits. The cumulative frequency gives the total number of unique subjects which is 470 as we have already observed.

Another task that you may wish to perform on a dataset with multiple observations is to create a dataset that contains only the first observation for each subject. Such a dataset could then be used to generate statistics on the distribution of subject characteristics at baseline of the study. The first step in this process is to sort the dataset on the ID and secondarily on the visit date. The following program accomplishes this sort.

```
PROC SORT DATA=TEMP;
    BY ID VISDATE;
RUN;
```

Let's take a look at the dataset with the Viewtable. Notice how the first visit for each subject is now listed first. This first visit is the one that we would like to output into another dataset so that we can analyze the baseline characteristics of our cohort of subjects.

SAS has a couple of internal variables that are very useful for this purpose called *FIRST.variable* and *LAST.variable*. These internal variables are available **only** when you are using a BY statement in a DATA step. In a dataset with multiple observations with the same value in one of the variables, the FIRST variable is given the value of 1 for the first instance of the value and 0 for every other instance. In contrast, the variable LAST has the value of 1 for the last occurrence of the value and 0 otherwise. Let's see how we can use these internal variables to create a dataset that includes only the first observation for a subject. We will then do a frequency procedure on the variable EMPFT which is coded 1 if the subject is employed full-time and 0 otherwise.

```
DATA FIRSTVISIT;
    SET TEMP;
    BY ID;
    IF FIRST.ID=1;
RUN;

PROC FREQ DATA=FIRSTVISIT;
    TABLES EMPFT;
    TITLE Employment status at study baseline;
RUN;
```

If you wanted data from the last visit (e.g., you wanted to analyze data collected at the most recent visit to the clinic), you could use the same program but substitute the LAST variable for the FIRST variable. This program illustrates just one of the many uses for the *FIRST.variable* and *LAST.variable* internal variables in SAS. In order to use these variables, the dataset must be sorted on the variable that you wish to use these internal variables for and they must be used as part of a data step that processes the dataset using a BY statement.

Another common task that you may want to perform on datasets with multiple observations for the same subject is to create variables that contain one summary value regarding some characteristic that might change from one visit to the next. You would want to have this summary variable in a dataset that has just one observation per subject such as the FIRSTVISIT dataset that we created earlier. This would allow you to analyze these summary variables across the study subjects who participated in the study.

For example, the dataset PATIENTS contains a variable called ALLBLDS that records the number of bleeding episodes experienced by the subject since the last clinic visit. An example of a summary variable that we might like to create is the average number of bleeding episodes experienced by the

subject during the study follow up period. The program below illustrates how PROC SUMMARY can be used to create this variable.

```
PROC SUMMARY DATA=TEMP2;
  CLASS ID;
  VAR ALLBLDS;
  OUTPUT OUT=TEMP4 MEAN=AVGBLDS;
RUN;
```

Let's take a look at the distribution of the AVGBLDS variable using the UNIVARIATE procedure. Remember that you should not include the first observation in the TEMP4 dataset because it is the sum of all of the values for AVGBLDS.

```
PROC UNIVARIATE DATA=TEMP4;
  VAR AVGBLDS;
  WHERE _TYPE_=1;
  TITLE; *This line turns off the title from the previous frequency procedure;
RUN;
```

Now that you have this average number of bleeds value for each observation in your dataset you can merge it into your database that contains only the data collected at the first visit. You can do that using the following program.

```
PROC SORT DATA=FIRSTVISIT;
  BY ID;
RUN;

PROC SORT DATA=TEMP4;
  BY ID;
RUN;

DATA TEMP5(DROP=_TYPE_ _FREQ_);
  MERGE FIRSTVISIT(IN=A) TEMP4;
  BY ID;
  IF A;
RUN;
```

You can create multiple summary variables in one data step using the summary procedure. For example, in addition to the average number of total bleeds we can create a variable for the average number of joint bleeds by using the JTBLDS variable. Additionally, let's say we want to create a variable that is 1 if the subject was employed full time at any visit and 0 if they were not employed full time at any visit. We can use the EMPFT variable which is coded 1 if employed full time and 0 if not. By taking the maximum value of EMPFT across all visits, we will have an indicator for employment at any visit. This works for any variable coded 1 for yes and 0 for no.

```
PROC SUMMARY DATA=TEMP2;
  CLASS ID;
  VAR ALLBLDS JTBLDS EMPFT;
  OUTPUT OUT=TEMP4 MEAN(ALLBLDS JTBLDS)=AVGBLDS AVGJTBLDS
```

```

MAX(EMPFT)=EVEREMPFT;
RUN;

```

Again, you can merge this into your database that contains only the data collected at the first visit.

```

PROC SORT DATA=FIRSTVISIT;
  BY ID;
RUN;

PROC SORT DATA=TEMP4;
  BY ID;
RUN;

DATA TEMP5(DROP=_TYPE_ _FREQ_);
  MERGE FIRSTVISIT(IN=A) TEMP4;
  BY ID;
  IF A;
RUN;

```

Notice that when you are requesting more than one summary measure you indicate which variables that you want to summarize in parentheses and then provide the names of the variables that you wish SAS to store those summary values in after the equals sign. Just as before you can then merge these new summary variables into your dataset with just one observation per person.

One more example of how to use the *FIRST.variable* and *LAST.variable* internal SAS variables is creating a dataset that contains any duplicate records in a dataset that is not supposed to have them. To illustrate how this is done let's take a look at the dataset mod9. Several people in the dataset have been given the same value for STUDYID. First, perform a frequency count on the STUDYID variable.

```

PROC FREQ DATA=NICOLE.mod9;
  TABLES STUDYID;
RUN;

```

You can see that there are several instances in which the same STUDYID is used more than once. You would like to get all instances of these possible duplicate records into one dataset so that you can look at the data and try to determine if they are truly duplicate records or if the same STUDYID has been used for more than one patient. The following code uses the *FIRST* and *LAST* internal variables to perform this step very simply.

First the data must be sorted on the variable of interest.

```

PROC SORT DATA=NICOLE.mod9;
  BY STUDYID;
RUN;

```

Next we create the dataset containing any duplicate records.

```

DATA DUPES;
  SET NICOLE.mod9;

```

```

        BY STUDYID;
        IF ^(FIRST.STUDYID AND LAST.STUDYID);
RUN;

```

If you now take a look at the DUPES dataset, you will find that it contains all of the data for subjects with the same values for STUDYID. The way that this works is that the ^ symbol tells SAS that you want any observations in which the FIRST and LAST internal variables are not equal to 1. In a dataset that contains only one instance of a particular STUDYID, both the FIRST and LAST variables will always be equal to 1. If there is more than one instance of a STUDYID, this will not be the case and the code above tells SAS to copy these observations into the data subset called DUPES.

9.2 - Other types of repeated observations (one more example using FIRST.variable and LAST.variable)

The dataset FLBYGROUP.SAS7BDAT has variables for STUDYID, SITE, and AGEGROUP. In this example we don't have repeated rows for STUDYID, but instead we have repeated values for SITE and AGEGROUP.

Let's say we want to count up the total number of records for EACH AGEGROUP within EACH SITE.

First, we would sort the NICOLE.FLBYGROUP dataset by SITE and AGEGROUP.

```

PROC SORT DATA = NICOLE.FLBYGROUP; by SITE AGEGROUP; RUN;

```

Next, we can ask SAS to count using these by variables, so SAS will increment by 1 on each row for a type (by site and agegroup) until it hits a new site and agegroup combo, then start a new counter.

```

DATA FLBYGROUP_REV;
    SET NICOLE.FLBYGROUP;
        COUNT + 1;
    BY SITE AGEGROUP;
    IF FIRST.AGEGROUP then COUNT=1;
RUN;

```

If we open the table, we can see that SAS counted a total of 17 with site=atlanta and agegroup=1, and 12 with site=atlanta and agegroup=2.

	studyid	site	agegroup	count
1	1001	atlanta	1	1
2	1012	atlanta	1	2
3	1015	atlanta	1	3
4	1020	atlanta	1	4
5	1023	atlanta	1	5
6	1031	atlanta	1	6
7	1034	atlanta	1	7
8	1036	atlanta	1	8
9	1038	atlanta	1	9
10	1047	atlanta	1	10
11	1059	atlanta	1	11
12	1070	atlanta	1	12
13	1071	atlanta	1	13
14	1074	atlanta	1	14
15	1075	atlanta	1	15
16	1099	atlanta	1	16
17	1100	atlanta	1	17
18	1009	atlanta	2	1
19	1013	atlanta	2	2
20	1040	atlanta	2	3
21	1051	atlanta	2	4
22	1053	atlanta	2	5
23	1054	atlanta	2	6
24	1058	atlanta	2	7
25	1061	atlanta	2	8
26	1062	atlanta	2	9
27	1091	atlanta	2	10
28	1092	atlanta	2	11
29	1095	atlanta	2	12

So now we can use `LAST.variable` to keep only the final answer for all the combos. *We are already sorted but could sort again by SITE and AGEGROUP to be sure.*

```
DATA FLBYGROUP_REV2;
    SET FLBYGROUP_REV;
    BY SITE AGEGROUP;
    IF LAST.AGEGROUP;
RUN;
```

	studyid	site	agegroup	count
1	1100	atlanta	1	17
2	1095	atlanta	2	12
3	1084	augusta	1	6
4	1093	augusta	2	8
5	1097	decatur	1	13
6	1094	decatur	2	13
7	1096	marietta	1	6
8	1090	marietta	2	9
9	1098	savannah	1	5
10	1087	savannah	2	11

9.3 - Dealing with duplicates

Another handy tool for dealing with duplicates is within the SORT procedure itself. In the code below, the same mod9 dataset is sorted by STUDYID. The NODUPKEY statement removes duplicate values, leaving just one record per STUDYID. The OUT statement produces a dataset called NODUPES which has the de-duplicated records only. This is very important since using NODUPKEY without specifying an OUT dataset will remove the duplicate records from the dataset you are sorting. The DUPOUT statement places the duplicate records that were removed into a separate dataset called DUPESONLY that you can review. This procedure might be best when you know the rows are true duplicates – in large datasets with a lot of variables it might be best to use a flag like we did in the step above so you can carefully review any duplicate records and decide which to remove.

```
PROC SORT DATA= NICOLE.mod9 NODUPKEY OUT=NODUPES DUPOUT=DUPESONLY;
    BY STUDYID;
RUN;
```

9.4 - Using the RETAIN statement

The RETAIN statement is used to “save” a value from one observation to the next. It has many uses.

First, let’s do a basic example. We will create a dataset with variables for month and income.

```
data retain_ex1_a;
input month income;
datalines;
1 12451
2 54325
3 43514
4 13455
5 45161
6 54151
7 54261
8 43251
9 43515
10 83711
11 45236
12 54361
;
run;
```

If we try to create a variable to store the sum of income as a cumulative total without using the retain statement, we get the following dataset where the cumulative income is just equal to income on each row. This is because we didn’t tell SAS to retain the value from the previous row and add the new month value to the previous month value each time.

```
data retain_ex1_b;
    set retain_ex1_a;
    cumulative_income = sum(income,cumulative_income);
run;
```


	month	income	cumulative_income
1	1	12451	12451
2	2	54325	54325
3	3	43514	43514
4	4	13455	13455
5	5	45161	45161
6	6	54151	54151
7	7	54261	54261
8	8	43251	43251
9	9	43515	43515
10	10	83711	83711
11	11	45236	45236
12	12	54361	54361

Now, let's try it with RETAIN.

```
data retain_ex1_c;
    set retain_ex1_a;
    retain cumulative_income;
    cumulative_income = sum(income,cumulative_income);
run;
```

	month	income	cumulative_income
1	1	12451	12451
2	2	54325	66776
3	3	43514	110290
4	4	13455	123745
5	5	45161	168906
6	6	54151	223057
7	7	54261	277318
8	8	43251	320569
9	9	43515	364084
10	10	83711	447795
11	11	45236	493031
12	12	54361	547392

Now, let's do a slightly more complicated example that includes BY groups. Create a dataset from the SAS help files using the following code:

```
data prdsale;
    set sashelp.prdsale;
    where country = 'CANADA' and product = 'SOFA';
    drop region division prodtype predict quarter;
run;
```

To visualize/review just what FIRST.variable and LAST.variable are doing, we will create a new variable called month_order.

AEPI537D

```
proc sort data = prdsale;
    by month actual ;
run;

data first_last;
    set prdsale;
    by month;

    if first.month then month_order = 'FIRST';
    if last.month then month_order = 'LAST';
run;
```

	Actual Sales	Country	Product	Year	Month	month_order
1	\$5.00	CANADA	SOFA	1993	Jan	FIRST
2	\$72.00	CANADA	SOFA	1993	Jan	
3	\$925.00	CANADA	SOFA	1993	Jan	
4	\$951.00	CANADA	SOFA	1993	Jan	LAST
5	\$164.00	CANADA	SOFA	1993	Feb	FIRST
6	\$394.00	CANADA	SOFA	1993	Feb	
7	\$926.00	CANADA	SOFA	1993	Feb	
8	\$999.00	CANADA	SOFA	1993	Feb	LAST
9	\$422.00	CANADA	SOFA	1993	Mar	FIRST
10	\$608.00	CANADA	SOFA	1993	Mar	
11	\$615.00	CANADA	SOFA	1993	Mar	
12	\$850.00	CANADA	SOFA	1993	Mar	LAST

Now we will calculate cumulative totals within BY groups. In this example, we want to create a cumulative total of Actual Sales for EACH month. Meaning each row listed as month of 'Jan' should continue to be added until a new 'Feb' is reached, and so on.

First we will demonstrate doing this without using RETAIN. As you can see, it only put the total on the first row for each month.

```
proc sort data = prdsale;
    by month ;
run;
data prdsale_noretain;
    set prdsale;
    by month;

    if first.month then cumulative_actual = actual;
    else cumulative_actual = cumulative_actual + actual;
run;
```

AEPI537D

	Actual Sales	Country	Product	Year	Month	cumulative_actual
1	\$5.00	CANADA	SOFA	1993	Jan	5
2	\$72.00	CANADA	SOFA	1993	Jan	.
3	\$925.00	CANADA	SOFA	1993	Jan	.
4	\$951.00	CANADA	SOFA	1993	Jan	.
5	\$164.00	CANADA	SOFA	1993	Feb	164
6	\$394.00	CANADA	SOFA	1993	Feb	.
7	\$926.00	CANADA	SOFA	1993	Feb	.
8	\$999.00	CANADA	SOFA	1993	Feb	.
9	\$422.00	CANADA	SOFA	1993	Mar	422
10	\$608.00	CANADA	SOFA	1993	Mar	.
11	\$615.00	CANADA	SOFA	1993	Mar	.
12	\$850.00	CANADA	SOFA	1993	Mar	.

Now, using RETAIN. You can see that SAS saved the values between rows of the same month to create a cumulative total by month.

```
proc sort data = prdsale;
    by month ;
run;
data prdsale_retain;
    set prdsale;
    by month;

    retain cumulative_actual;

    if first.month then cumulative_actual = actual;
    else cumulative_actual = cumulative_actual + actual;

run;
```

	Actual Sales	Country	Product	Year	Month	cumulative_actual
1	\$5.00	CANADA	SOFA	1993	Jan	5
2	\$72.00	CANADA	SOFA	1993	Jan	77
3	\$925.00	CANADA	SOFA	1993	Jan	1002
4	\$951.00	CANADA	SOFA	1993	Jan	1953
5	\$164.00	CANADA	SOFA	1993	Feb	164
6	\$394.00	CANADA	SOFA	1993	Feb	558
7	\$926.00	CANADA	SOFA	1993	Feb	1484
8	\$999.00	CANADA	SOFA	1993	Feb	2483
9	\$422.00	CANADA	SOFA	1993	Mar	422
10	\$608.00	CANADA	SOFA	1993	Mar	1030
11	\$615.00	CANADA	SOFA	1993	Mar	1645
12	\$850.00	CANADA	SOFA	1993	Mar	2495

Now, let's do one EVEN MORE complicated example.

One of the most common is when dealing with datasets with multiple records per person. In the PATIENTS dataset, we may want to create a variable HXJTBlds for history of joint bleeds (whether the person has ever had a joint bleed in the past) when all we have currently is whether or not the person reported a joint bleed at the current study visit (JTBlds).

	id	visdate	jtbls
157	052-2733-85482	22SEP1998	0
158	052-2733-85482	12OCT1999	14
159	052-2733-85482	19JUN2001	10
160	052-2733-85482	12MAR2002	0
161	052-2734-46966	22SEP1998	2
162	052-2734-46966	12OCT1999	0
163	052-2734-46966	19SEP2000	0
164	052-2734-46966	12MAR2002	0
165	052-2734-46966	03JUN2003	12
166	053-0398-77614	24AUG1998	16
167	053-0398-77614	18OCT1999	12
168	053-0398-77614	30OCT2000	12
169	053-0398-77614	03DEC2001	14
170	053-0398-77614	03MAR2003	20
171	053-0443-12519	29MAR1999	12

Let's look at the individual on row 157 (ID # 052-2733-85482) using the sorted TEMP dataset we made earlier.

This person attended 4 study visits and reported joint bleeds (JTBLDS) at visits 2 and 3, but not at visits 1 and 4.

We should consider this person to have a history of joint bleeds at visits 2, 3 and 4 (as soon as this person had his or her first joint bleed, he or she is now considered to have a history of the condition).

We would like to create our history variable, HXJTBLDS, so that it assigns a value of 0 to the first visit, and a value of 1 to the second, third and fourth visits.

First we will create a new dataset and like before, sort it by ID and VISDATE so the dataset is ordered by person, and within each person, by visit date.

```
DATA TEMP6;
    SET NICOLE.PATIENTS;
RUN;

PROC SORT DATA = TEMP6;
    BY ID VISDATE;
RUN;
```

Now we will write the code to create the new variable HXJTBLDS. The code is more complicated that what we have been doing to date so comments are included before each section of code to explain the purpose.

```
*Create the new dataset;
DATA TEMP7;
    SET TEMP6;

*Use the BY statement so we will be able to use the FIRST.ID function later on;
BY ID;

*Create a new dichotomous variable for JTBLDS called EVERJTBLDS that will have a
value of 1 if the person reported one or more joint bleeds at the study visit and 0
if they reported none. To simplify the code, we will assume there are no missing
values ;
IF JTBLDS > 0 THEN EVERJTBLDS = 1;
ELSE IF JTBLDS = 0 THEN EVERJTBLDS = 0;
```

```
*Look at the first visit for the person to decide what the "baseline" history of
joint bleeds (history at the first visit) is for this person;
IF FIRST.ID THEN HXJTBlds = EVERJTBlds;
```

```
*If this is not the first visit, we will need to look at the previous visit to
decide what the history of joint bleeds is. If the history was negative but the
person now reports a joint bleed, we will have to count this person as having a
history of joint bleeds, otherwise we will just assign the previous history
(KEEPJTBlds);
ELSE DO;
IF EVERJTBlds = 1 THEN HXJTBlds = 1;
ELSE HXJTBlds = KEEPJTBlds;
END;
```

```
*You will notice that above we introduced a new variable, KEEPJTBlds (it is a dummy
variable that is a duplicate of HXJTBlds, as shown below). This variable is defined
below the ELSE statement to take the value of the updated history of joint bleeds
(HXJTBlds). The RETAIN statement is used so that our variable KEEPJTBlds is saved
in SAS memory;
KEEPJTBlds = HXJTBlds;
RETAIN KEEPJTBlds;
```

```
RUN;
```

You may be wondering why KEEPJTBlds is only defined *after* it is used, which would normally result in an error. But in fact if you follow how SAS reads the code, it is actually defined *before*.

SAS reads the *first* observation of the dataset. At the IF-ELSE statement, it will perform the code for IF FIRST.ID THEN HXJTBlds = EVERJTBlds and will ignore the ELSE statement. Then it will proceed to the KEEPJTBlds = HXJTBlds line of code and the RETAIN statement.

On the *second* and subsequent observations for the person, when SAS encounters the IF-ELSE statement, it chooses the ELSE option. By this time though, SAS has KEEPJTBlds in memory (because of the RETAIN statement from its last read-through) and so it uses this value in the code.

We typically want to have the RETAIN statement near the end of our code so that the most up-to-date variable gets saved into memory for the start of the next observation.

	id	visdate	jtbls	everjtbls	hxjtbls	keepjtbls
157	052-2733-85482	22SEP1998	0	0	0	0
158	052-2733-85482	12OCT1999	14	1	1	1
159	052-2733-85482	19JUN2001	10	1	1	1
160	052-2733-85482	12MAR2002	0	0	1	1
161	052-2734-46966	22SEP1998	2	1	1	1
162	052-2734-46966	12OCT1999	0	0	1	1
163	052-2734-46966	19SEP2000	0	0	1	1
164	052-2734-46966	12MAR2002	0	0	1	1
165	052-2734-46966	03JUN2003	12	1	1	1
166	053-0398-77614	24AUG1998	16	1	1	1

We can see from the new dataset that the HXJTBlds variable now correctly codes the history of joint bleeds in the first individual (compare EVERJTBlds to HXJTBlds).

We can also see for the next person (ID # 052-2734-46966) that the history of joint bleeds was also coded correctly; this person reported joint bleeds at the first and fifth study visits, so was coded as having a history of joint bleeds starting at the first visit.

KEEPJTBlds is a dummy variable that is a duplicate of HXJTBlds and used only for the RETAIN statement. It can be dropped from the final dataset (be careful not to drop the variable during the DATA step or it will not be retained through the next observation!). Creating this dummy variable is likely not necessary (you can choose to retain HXJTBlds instead); however, creating a new variable can be used as insurance in case your code goes wrong – you can compare the HXJTBlds and KEEPJTBlds variables to make sure the right values are being retained (whereas when using only one variable you have nothing to compare to).

MOD 9: USE SAS COMMENTS TO NUMBER YOUR WORK.

Using the dataset PATIENTS_PART2.sas7bdat:

1. Write a program that will allow you to determine how many unique patients there are in the dataset.
2. Write a program that creates a variable called ALLVISITS in a temporary SAS dataset called TEMP2 that stores the total number of visits for each patient.
3. Write a program that will allow you to use a SAS procedure to get the average number of joint bleeds (JTBLDS) for all patients at the MOST RECENT visit. Hint: Create a SAS dataset that contains data collected on subjects at their last visit only.
4. Write a program to create a new variable that will store the average weight for each subject (WEIGHTKG) over the study period. Also write a program that brings this new variable into the dataset that you created to answer question 3.

Using the dataset SCREENER_DATA.sas7bdat:

5. Write a program that calculates the total number of records (all screening records in the file), the total number of partial and complete records (Vstatus), and the total number of complete records that are eligible and ineligible (ELIG) for each site (SITE).

Vstatus is coded as "Complete" or "Partial"

ELIG is coded as 0 (no) or 1 (yes)

SITE is coded as "City A" or "City B"